

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

по лабораторной работе № 7

дисциплина: Архитектура компьютера

Студент: Венис Дмитрий Сергеевич

Группа: НКАбд-01-25

МОСКВА

2025 г.

Содержание

1. Цель работы
2. Теоретическое введение
3. Выполнение лабораторной работы
4. Выполнение самостоятельных заданий
5. Выводы

Список иллюстраций:

Рис.3.1.....	7
Рис.3.2.....	7
Рис.3.3.....	8
Рис.3.4.....	8
Рис.3.5.....	9
Рис.3.6.....	9
Рис.3.7.....	9
Рис.3.8.....	9
Рис.3.9.....	9
Рис.3.10.....	10
Рис.3.11.....	10
Рис.3.12.....	10
Рис.3.13.....	12
Рис.3.14.....	12
Рис.4.1.....	13
Рис.4.2.....	13
Рис.4.3.....	14
Рис.4.4.....	14
Рис.4.5.....	14
Рис.4.6.....	15

1.Цель работы

Изучение команд условного и безусловного переходов. Приобретение навыков написания программ с использованием переходов. Знакомство с назначением и структурой файла листинга.

2. Теоретическое введение

Для реализации ветвлений в ассемблере используются так называемые команды передачи управления или команды перехода. Можно выделить 2 типа переходов:

- **условный переход** – выполнение или не выполнение перехода в определенную точку программы в зависимости от проверки условия.
- **безусловный переход** – выполнение передачи управления в определенную точку программы без каких-либо условий.

2.1. Команды безусловного перехода

Безусловный переход выполняется инструкцией `jmp` (от англ. `jump` – прыжок), которая включает в себя адрес перехода, куда следует передать управление:

`jmp <адрес_перехода>`

Адрес перехода может быть либо меткой, либо адресом области памяти, в которую предварительно помещен указатель перехода. Кроме того, в качестве операнда можно использовать имя регистра, в таком случае переход будет осуществляться по адресу, хранящемуся в этом регистре.

В следующем примере рассмотрим использование инструкции `jmp`:

`label:`

`... ;`

`... ; команды`

`... ;`

`jmp label`

2.2. Команды условного перехода

Как отмечалось выше, для условного перехода необходима проверка какого-либо условия. В ассемблере команды условного перехода вычисляют условие перехода анализируя флаги из регистра флагов.

2.2.1. Регистр флагов

Флаг – это бит, принимающий значение 1 («флаг установлен»), если выполнено некоторое условие, и значение 0 («флаг сброшен») в противном случае. Флаги работают независимо друг от друга, и лишь для удобства они помещены в единый регистр — регистр флагов, отражающий текущее состояние процессора. В следующей таблице указано положение битовых флагов в регистре флагов.

2.2.2. Описание инструкции *сmp*

Инструкция *сmp* является одной из инструкций, которая позволяет сравнить операнды и выставляет флаги в зависимости от результата сравнения.

Инструкция *сmp* является командой сравнения двух операндов и имеет такой же формат, как и команда вычитания:

Сmp <операнд_1>, <операнд_2>

Команда *сmp*, так же как и команда вычитания, выполняет вычитание - , но результат вычитания никуда не записывается и единственным результатом команды сравнения является формирование флагов.

2.2.3. Описание команд условного перехода.

Команда условного перехода имеет вид

j <мнемоника перехода> label

Мнемоника перехода связана со значением анализируемых флагов или со способом формирования этих флагов.

Представлены команды условного перехода, которые обычно ставятся после команды сравнения `сmp`. В их мнемокодах указывается тот результат сравнения, при котором надо делать переход. Мнемоники, идентичные по своему действию, написаны в таблице через дробь (например, `ja` и `jnbe`). Программист выбирает, какую из них применить, чтобы получить более простой для понимания текст программы.

3.Выполнение лабораторной работы

1. Создадим каталог для программ лабораторной работы № 7, перейдем в него и создадим файл lab7-1.asm:

```
venisdmitry@fedora:~/project/HelperBot$ mkdir ~/work/arch-pc/lab07
venisdmitry@fedora:~/project/HelperBot$ cd ~/work/arch-pc/lab07
venisdmitry@fedora:~/work/arch-pc/lab07$ touch lab7-1.asm
venisdmitry@fedora:~/work/arch-pc/lab07$
```

Рис.3.1

2.Введем в файл lab7-1.asm текст программы из листинга 7.1.

```
lab7-1.asm      [----] 41 L:[ 1+19 20/ 20] *(649 / 649b) <EOF>      [*][X]
%include 'in_out.asm' ; подключение внешнего файла
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start
_start:
jmp _label2
_label1:
mov eax, msg1 ; Вывод на экран строки
call sprintf ; 'Сообщение № 1'
_label2:
mov eax, msg2 ; Вывод на экран строки
call sprintf ; 'Сообщение № 2'
_label3:
mov eax, msg3 ; Вывод на экран строки
call sprintf ; 'Сообщение № 3'
_end:
call quit ; вызов подпрограммы завершения
```

Рис.3.2

Создадим исполняемый файл и запустим его. Результат работы данной программы будет следующим:


```

venisdmitry@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
venisdmitry@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
venisdmitry@fedora:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 2
Сообщение № 3
venisdmitry@fedora:~/work/arch-pc/lab07$

```

Рис.3.3

Изменим текст программы в соответствии с листингом 7.2.

Создадим исполняемый файл и проверим его работу. Изменим текст программы добавив или изменив инструкции `jmp`, чтобы вывод программы был следующим:

user@dk4n31:~\$./lab7-1

Сообщение № 3

Сообщение № 2

Сообщение № 1

user@dk4n31:~\$

```

lab7-1.asm  [----] 30 L: [ 1+19 20/ 23] *(594 / 682b) 0039 0x027  [*][X]
#include 'in_out.asm' ; подключение внешнего файла
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start
_start:
jmp _label3
_label1:
mov eax, msg1 ; Вывод на экран строки
call sprintf ; 'Сообщение № 1'
jmp _end
_label2:
mov eax, msg2 ; Вывод на экран строки
call sprintf ; 'Сообщение № 2'
jmp _label1
_label3:
mov eax, msg3 ; Вывод на экран строки
call sprintf ; 'Сообщение № 3'
jmp _label2
_end:
call quit ; вызов подпрограммы завершения

```

Рис.3.4

```

venisdmity@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
venisdmity@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
venisdmity@fedora:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 3
Сообщение № 2
Сообщение № 1
venisdmity@fedora:~/work/arch-pc/lab07$

```

Рис.3.5

3.Создадим файл lab7-2.asm в каталоге ~/work/arch-pc/lab07. Внимательно изучим текст программы из листинга 7.3 и введем в lab7-2.asm

```

venisdmity@fedora:~/work/arch-pc/lab07$ touch lab7-2.asm

```

```

lab7-2.asm      [-M--] 17 L: 1+ 0 1/ 49] *(17 /1743b) 6097 8x861
%include "in_out.asm"
section .data
msg1 db "Введите B: ",0h
msg2 db "Наибольшее число: ",0h
A dd "20"
C dd "50"
section .bss
max resb 10
B resb 10
section .text
global _start
_start:
; ----- Вывод сообщения 'Введите B: '
mov eax,msg1
call sprint
; ----- Ввод 'B'
mov ecx,B
mov edx,10
call read
; ----- Преобразование 'B' из символа в число
mov eax,B
call atoi ; Вывод подпрограммы перевода символа в число
mov [B],eax ; запись преобразованного числа в 'B'
; ----- Записываем 'A' в переменную 'max'
mov ecx,[A] ; 'ecx = A'
mov [max],ecx ; 'max = A'
; ----- Сравниваем 'A' и 'C' (как символы)
cmp ecx,[C] ; Сравниваем 'A' и 'C'
jg check_B ; если 'A>C', то переход на метку 'check_B'
mov ecx,[C] ; иначе 'ecx = C'
mov [max],ecx ; 'max = C'
; ----- Преобразование 'max(A,C)' из символа в число
check_B:
mov eax,max
call atoi ; Вывод подпрограммы перевода символа в число
mov [max],eax ; запись преобразованного числа в 'max'
; ----- Сравниваем 'max(A,C)' и 'B' (как числа)
mov ecx,[max]
cmp ecx,[B] ; сравниваем 'max(A,C)' и 'B'
jg fin ; если 'max(A,C)>B', то переход на 'fin',
mov ecx,[B] ; иначе 'ecx = B'
mov [max],ecx
; ----- Вывод результата
fin:
mov eax,msg2
call sprint ; Вывод сообщения 'Наибольшее число: '
mov eax,[max]

```

```

; ----- Вывод результата
fin:
mov eax,msg2
call sprint ; Вывод сообщения 'Наибольшее число: '
mov eax,[max]
call printf ; Вывод 'max(A,B,C)'
call quit ; Выход

```

Рис.3.6,3.7,3.8

Создадим исполняемый файл и проверим его работу для разных значений B.

```

venisdmity@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-2.asm
venisdmity@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-2 lab7-2.o
venisdmity@fedora:~/work/arch-pc/lab07$ ./lab7-2
Введите B: 1
Наибольшее число: 50
venisdmity@fedora:~/work/arch-pc/lab07$

```

```
venisdmitry@fedora:~/work/arch-pc/lab07$ ./lab7-2
Введите В: 2
Наибольшее число: 50
venisdmitry@fedora:~/work/arch-pc/lab07$ ./lab7-2
Введите В: 45
Наибольшее число: 50
venisdmitry@fedora:~/work/arch-pc/lab07$
```

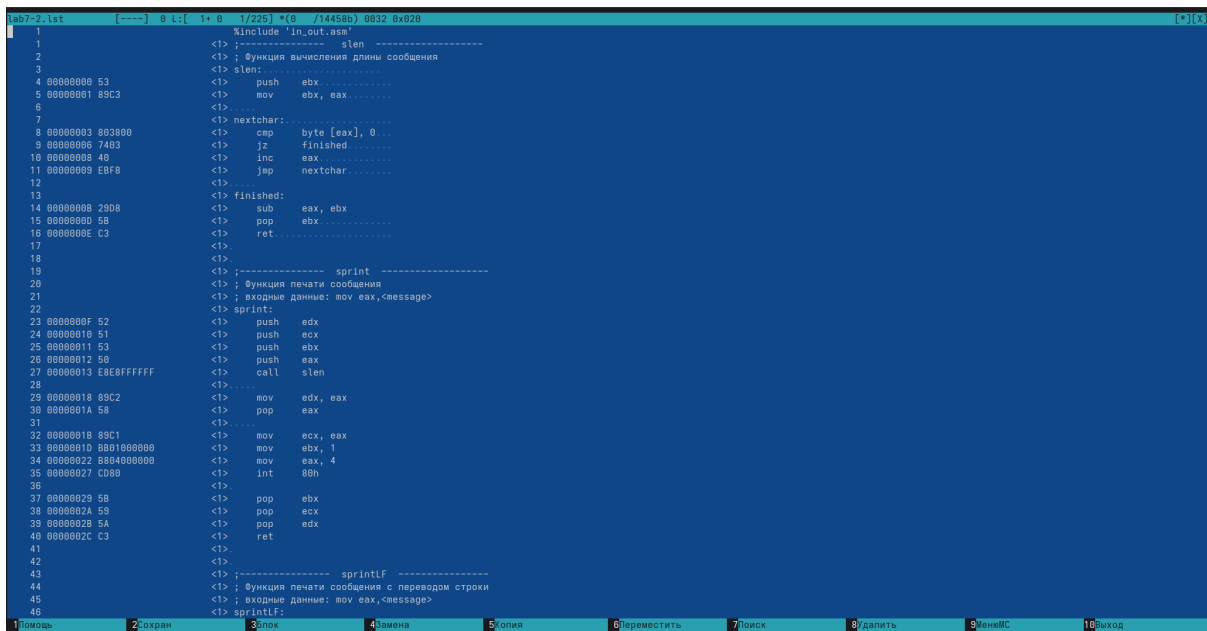
Рис.3.9,3.10

4.Создадим файл листинга для программы из файла lab7-2.asm

```
venisdmitry@fedora:~/work/arch-pc/lab07$ nasm -f elf -l lab7-2.lst lab7-2.asm
venisdmitry@fedora:~/work/arch-pc/lab07$
```

Рис.3.11

Откроем файл листинга lab7-2.lst с помощью любого текстового редактора, например mcedit



```
lab7-2.lst [----] 0 1: 1+ 8 1/225 *(0 /14458b) 0832 0x828
1
2 <I> ;----- slen -----
3 <I> ; функция вычисления длины сообщения
4 <I> slen:
5 00000000 53 <I> push ebx
6 00000001 89C3 <I> mov ebx, eax
7
8 00000003 803800 <I> nextchar:
9 00000000 7403 <I> cmp byte [eax], 0
10 00000000 40 <I> jz finished
11 00000000 EBF8 <I> inc eax
12 <I> jmp nextchar
13
14 00000000 20D0 <I> finished:
15 00000000 5B <I> sub eax, ebx
16 00000000 C3 <I> pop ebx
17 <I> ret
18
19 <I> ;----- sprint -----
20 <I> ; функция печати сообщения
21 <I> ; входные данные: mov eax,<message>
22 <I> sprint:
23 0000000F 52 <I> push edx
24 00000010 51 <I> push ecx
25 00000011 53 <I> push ebx
26 00000012 50 <I> push eax
27 00000013 E8BFFFFFFF <I> call slen
28
29 00000018 89C2 <I> mov edx, eax
30 0000001A 58 <I> pop eax
31 <I>
32 0000001B 89C1 <I> mov ecx, eax
33 0000001D 80100000 <I> mov ebx, 1
34 00000022 B804000000 <I> mov eax, 4
35 00000027 CD80 <I> int 80h
36
37 00000029 59 <I> pop ebx
38 0000002A 59 <I> pop ecx
39 0000002B 5A <I> pop edx
40 0000002C C3 <I> ret
41
42 <I> ;----- sprintf -----
43 <I> ; функция печати сообщения с переводом строки
44 <I> ; входные данные: mov eax,<message>
45 <I> sprintf:
46 <I>
```

Рис.3.12

5 строка:

00000001 89C3 mov ebx, eax

эта строка выполняет сохранение начального адреса строки перед циклом; инструкция копирует текущее значение из регистра *eax* (указатель на начало сообщения) в регистр *ebx*, что необходимо для дальнейшего вычисления длины строки путем вычитания этого сохраненного адреса из конечного адреса после прохождения цикла.

8 строка:

00000003 803800 cmp byte [eax], 0

эта строка осуществляет проверку условия выхода из цикла; инструкция сравнивает значение байта, находящегося в памяти по адресу из регистра *eax*, с нулем (нуль-терминатором), и если они равны, процессор выставляет флаг нуля (ZF) в регистре флагов в единицу.

9 строка:

00000006 7403 jz finished

эта строка реализует условное ветвление программы; инструкция анализирует флаг нуля (ZF), и если он установлен (то есть был найден конец строки), выполнение программы перенаправляется на метку *finished* (смещение +3 байта), пропуская тело цикла.

Откроем файл с программой lab7-2.asm и в любой инструкции с двумя операндами, удалим один операнд. Выполним трансляцию с получением файла листинга.

```

dmitryvenis@work/arch-pc/lab7-2.asm: [-----] 7 L: [ 1+13 14/ 49] * (277 / 1738b) 0010 0x00A [*] (*)
include "in.out.asm"
section .data
msg1 db "Введите B: ",0h
msg2 db "Наибольшее число: ",0h
A dd '20'
C dd '50'
section .bss
max resb 10
B resb 10
section .text
global _start
_start:
; ----- Вывод сообщения 'Введите B: '
mov eax,1
call sprint
; ----- Ввод 'B'
mov ecx,B
mov edx,10
call sread
; ----- Преобразование 'B' из символа в число
mov eax,B
call atoi ; Вызов подпрограммы перевода символа в число
mov [B],eax ; запись преобразованного числа в 'B'
; ----- Записываем 'A' в переменную 'max'
mov ecx,[A] ; 'ecx = A'
mov [max],ecx ; 'max = A'
; ----- Сравниваем 'A' и 'C' (как символы)
cmp ecx,[C] ; Сравниваем 'A' и 'C'
jg check_B ; если 'A>C', то переход на метку 'check_B',
mov ecx,[C] ; иначе 'ecx = C'
mov [max],ecx ; 'max = C'
; ----- Преобразование 'max(A,C)' из символа в число
check_B:
mov eax,max
call atoi ; Вызов подпрограммы перевода символа в число
mov [max],eax ; запись преобразованного числа в 'max'
; ----- Сравниваем 'max(A,C)' и 'B' (как числа)
mov ecx,[max]
cmp ecx,[B] ; Сравниваем 'max(A,C)' и 'B'
jg fin ; если 'max(A,C)>B', то переход на 'fin',
mov ebx,[B] ; иначе 'ecx = B'
mov [max],ecx
; ----- Вывод результата
; -----
1|help 2|save 3|mark 4|replac 5|copy 6|move 7|search 8|delete 9|pullDn 10|quit

```

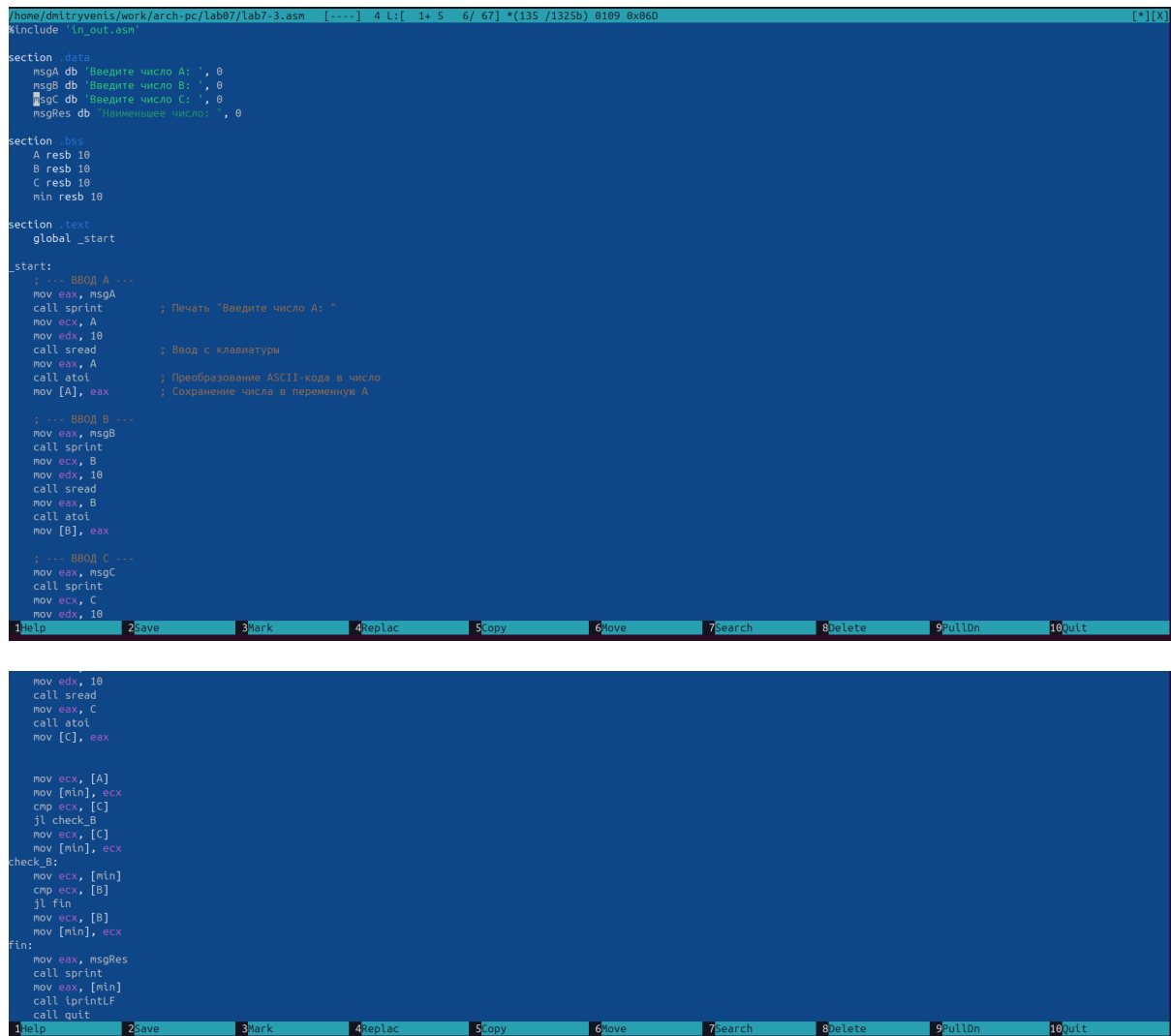
Рис.3.13,3.14

Появляется ошибка на экране, т.к. инструкция mov не может работать с одним аргументом, и, как следствие, транслятор не создаст новый файл листинга, поскольку листинг генерируется только при успешной компиляции без ошибок

4.Выполнение самостоятельной работы

1. Напишите программу нахождения наименьшей из 3 целочисленных переменных a, b и c . Значения переменных выбрать из табл. 7.5 в соответствии с вариантом, полученным при выполнении лабораторной работы № 7. Создайте исполняемый файл и проверьте его работу

Ответ:Напишем программу



```
home/cntrivents/work/arch-pc/lab07/lab7-3.asm  [----]  4  L:  1+ 5  6/ 67] *(135 /1325b) 0109 0x000  [*][X]
#include "in_out.asm"

section .data
    msgA db "Введите число A: ", 0
    msgB db "Введите число B: ", 0
    msgC db "Введите число C: ", 0
    msgRes db "Наименьшее число: ", 0

section .bss
    A resb 10
    B resb 10
    C resb 10
    min resb 10

section .text
    global _start

_start:
    ; --- БЛОК A ---
    mov eax, msgA
    call sprint
    mov ecx, A
    mov edx, 10
    call sread
    mov eax, A
    call atoi
    mov [A], eax
    ; Ввод с клавиатуры
    ; Преобразование ASCII-кода в число
    ; Сохранение числа в переменную A

    ; --- БЛОК B ---
    mov eax, msgB
    call sprint
    mov ecx, B
    mov edx, 10
    call sread
    mov eax, B
    call atoi
    mov [B], eax

    ; --- БЛОК C ---
    mov eax, msgC
    call sprint
    mov ecx, C
    mov edx, 10
    call sread
    mov eax, C
    call atoi
    mov [C], eax

    mov ecx, [A]
    mov [min], ecx
    cmp ecx, [C]
    jl check_B
    mov ecx, [C]
    mov [min], ecx

check_B:
    mov ecx, [min]
    cmp ecx, [B]
    jl fin
    mov ecx, [B]
    mov [min], ecx

fin:
    mov eax, msgRes
    call sprint
    mov eax, [min]
    call iprintf
    call quit
```

Рис.4.1,4.2

Создадим исполняемый файл и проверим его работу

```

dnitryenis@dnitryenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab07$ nasm -f elf lab7-3.asm
dnitryenis@dnitryenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-3 lab7-3.o
dnitryenis@dnitryenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab07$ ./lab7-3
Введите число A: 81
Введите число B: 22
Введите число C: 72
Наименьшее число: 22
dnitryenis@dnitryenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab07$

```

Рис.4.3

2. Напишите программу, которая для введенных с клавиатуры значений x и a вычисляет значение заданной функции $f(x)$ и выводит результат вычислений. Вид функции $f(x)$ выбрать из таблицы 7.6 вариантов заданий в соответствии с вариантом, полученным при выполнении лабораторной работы № 6. Создайте исполняемый файл и проверьте его работу для значений x и a из 7.6.

Ответ: Вариант 14

Напишем программу

```

dnitryenis@dnitryenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab07$ touch lab7-4.asm
dnitryenis@dnitryenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab07$

```

```

/home/dnitryenis/work/arch-pc/lab07/lab7-4.asm  [-M--] 13 L: 16+42 58/ 58) *(888 / 888b) <EOF>
_start:
    mov eax, msg_x
    call sprintf
    mov ecx, x
    mov edx, 80
    call sprintf
    mov eax, x
    call atoi
    mov [x], eax

    mov eax, msg_a
    call sprintf
    mov ecx, a
    mov edx, 80
    call sprintf
    mov eax, a
    call atoi
    mov [a], eax

    mov ecx, [x]
    cmp ecx, [a]
    jge greater_equal

    mov eax, [a]
    mov ebx, 3
    mul ebx
    inc ebx
    mov [res], eax
    jmp print_res

greater_equal:
    mov eax, [x]
    mov ebx, 3
    mul ebx
    inc ebx
    mov [res], eax

print_res:
    mov eax, msg_res
    call sprintf
    mov eax, [res]
    call printf
    call quit

```

Рис.4.4,4.5

Создадим исполняемый файл и проверим его работу для значений x и a

```
dmitryenis@dmitryenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab07$ nasm -f elf lab7-4.asm
dmitryenis@dmitryenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab07$ ld -n elf_i386 -o lab7-4 lab7-4.o
dmitryenis@dmitryenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab07$ ./lab7-4
Введите значение x: 1
Введите значение o: 2
Результат: 7
dmitryenis@dmitryenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab07$
```

Рис.4.6

Вывод:Выполнили самостоятельную работу

5.Выводы

Я изучил команды условного и безусловного переходов. Приобрел навыки написания программ с использованием переходов. Познакомился с назначением и структурой файла листинга.